南京信息工程大学　试卷

2021－2022 学年第 2 学期 数据结构 II 课程试卷(A 卷)

本试卷共 2 页；考试时间 100 分钟；任课教师 陈胜 ；出卷时间 2022 年 6 月

（闭卷考试，C 语言版）

学院 _____ 专业 _____ 班

学号 _____ 姓名 _____ 得分 _____

一、数据结构题(共 25 分)

1、用顺序表来存储字符串和用链表来存储字符串，讨论哪种存储方法更合理？（10 分）

2、写出数据元素为 int 型的单向链栈存储结构代码。（5 分）

3、图 2 是图 1 中有向图的存储示意图。写出相应存储结构代码。（10 分）

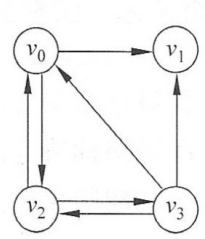


图 1. 有向图

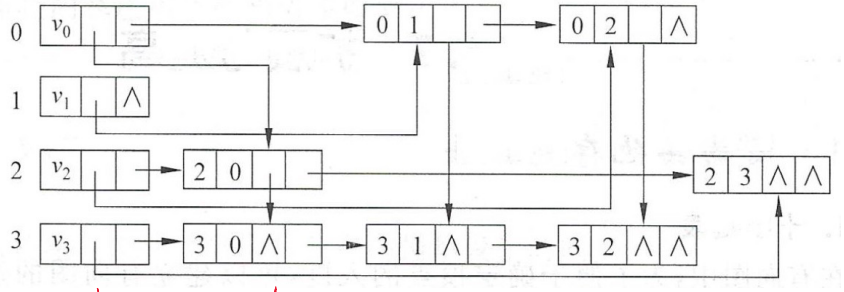


图 2. 有向图存储示意图

二、 算法基础题 (共 45 分)

1、画一棵结点为 ABCDEF 且后序遍历结果为 ABCDEF 的完全二叉树。（5 分）

2、画一棵结点权重分别为 10、15、20、25、30、40 的哈夫曼树。（5 分）

3、给出图 3 中结点 v_0 到结点 v_4 的最短路径及其路径长度。（5 分）

4、给出图 4 中事件 v_6 的最早发生时间和最迟发生时间。（6 分）

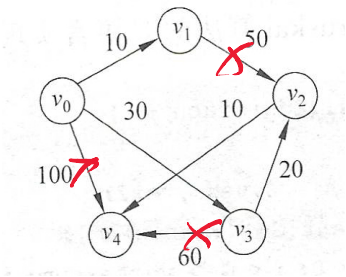


图 3　带权有向图

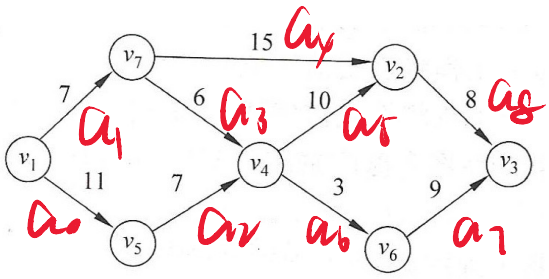


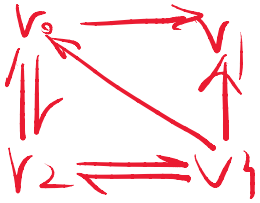
图 4　AOE 网

5、画一组二叉树图来给出查找集合{2,5,8,7,6}的平衡二叉树的构造过程。（6 分）

6、说明可以采用（不是如何采用）栈来实现图深度优先遍历算法的原因。（6 分）

7、对于关键字集{36,30,18,40,32,45,22,50}的堆排序，给出其初始建立的大根堆。（6 分）

8、给出希尔排序、起泡排序和归并排序这三种算法在平均情况下的时间复杂度。（6 分）



三、 综合能力题（共 30 分）

1、现欲设计一个算法，用来查找一个有向图中可能存在的有向回路，并且在存在的情况下，输出其中一个有向回路。用文字配合必要的代码，介绍自己所设计算法，并以图 1 中有向图为例，画出该算法中所采用的图存储结构示意图。（10 分）

2、图 5 中是图广度优先遍历算法的部分代码。阅读此部分代码后，完成下面问题：

（1）在图 5 中下划线处填写代码，使得填写后的代码可实现该遍历算法的功能；（10 分）

（2）用代码写出图 5 中 ALGraph 的存储结构定义；（8 分）

（3）在输入参数 v 对应图 1 中 v_0 的情况下，给出该算法对图 1 遍历的输出结果。（2 分）

```
01    void BFTraverse(ALGraph *G, int v)
02    {
03        EdgeNode *p = NULL;
04        int Q[MAXSIZE];
05        int front = rear = -1;
06        printf("%c", G->nodelist[v].label);
07        visited[v] = 1;
08        Q[++rear] = v;
09        while (front != rear)
10        {
11            v = Q[++front];
12            p = G->nodeList[j].first;
13            while (p != NULL)
14            {
15                j = p->adjIndex;
16                if (visited[j] == 0)
17                {
18                    printf("%c", G->nodeList[j].label);
19                    visited[j] = 1;
20                    Q[++rear] = j;
21                }
22                p = p->next;
23            }
24        }
25    }
```

图 5 图广度优先遍历算法部分代码

(2)
```
#define MAXSIZE 100
typedef char DataType;
typedef struct Edge
{
    int adjIndex;
    struct Edge* next;
}Edge;
typedef struct LabelNode
{
    DataType label;
    struct Edge* first;
}LabelNode;
typedef struct ALGraph
{
    LabelNode nodelist[MAXSIZE];
    int labelNum, EdgeNum;
}ALGraph;
```

label | first

adjIndex | next

(3). $V_0 V_1 V_2 V_3$

南京信息工程大学　试卷参考答案与评分标准

2021－2022 学年　第 2 学期　数据结构 II 课程　期末 A 卷

试卷共 2 页；考试时间 100 分钟；任课教师 陈胜 ；出卷时间 2022 年 6 月

一、 数据结构题

1、评分标准：

（1）顺序表存储更合理：应用场景 2 分，原因 2 分，缺点 1 分；（2）链表存储更合理：应用场景 2 分，原因 2 分，缺点 1 分。

2、所给代码的链结点中数据是 int 型元素（1 分）；所给代码的链结点含有指下一个结点的指针（2 分），和指向栈顶的指针（2 分）。参考答案：

```
typedef int DataType;
typedef struct Node
{
    DataType data;
    struct Node *next;
} *top;
```

3、Edge、Node 和 Graph 三个结构体各 3 分，其他 1 分。

参考答案：

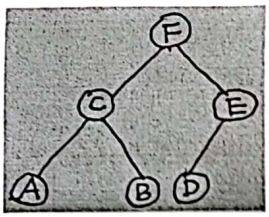
```
#define MAXSIZE 100;
typedef char[2] DataType;
typedef struct Edge
{
    int fromNode, toNode;
    struct Edge *sameFrom, *sameTo;
}
typedef struct Node
{
    DataType label;
    struct Edge *firstOut, *firstIn;
};
typedef struct Graph
{
    struct nodeList[MAXSIZE];
    int nodeNum, edgeNum;
}
```

Node *

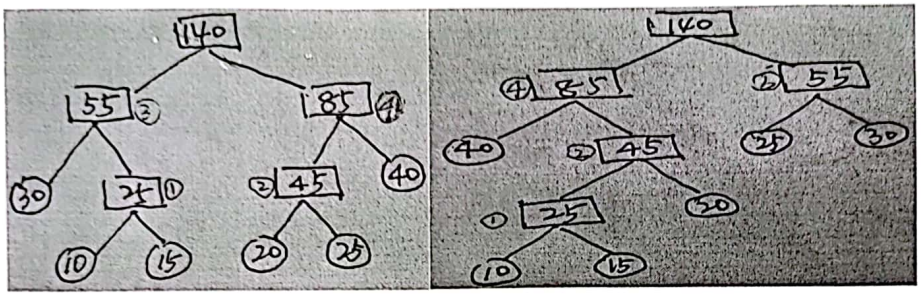
扫描全能王 创建

二、 算法基础题

1、评分标准（5 分）：完全二叉树结构 2 分，完全二叉树上标记 3 分。



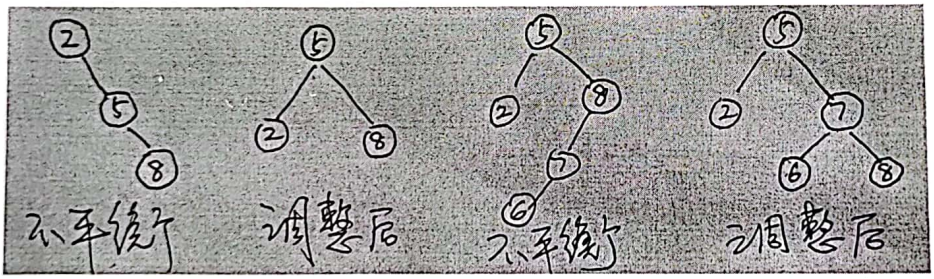
2、参考答案，下图中任意一个（5 分）：



3、参考答案（共 5 分），路径：v_0->v_3->v_2->v_4（共 4 分），路径长：60（共 1 分）。

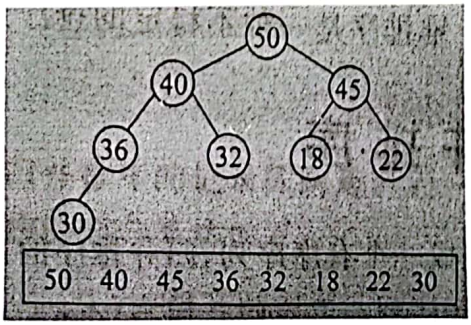
4、参考答案（共 6 分），事件 v_6 最早发生时间为 21（3 分），事件 v_6 最迟发生时间为 27（3 分），其中关键路径为 v_1->v_5->v_4->v_2 ->v_3，长度为 36。

5、参考答案（6 分）：两次的不平衡与其相应的调整，各 3 分。



6、评分标准（6 分）：
算法介绍正确（3 分）；
算法介绍沿先进后出或回溯的思路（2 分）；
结论 1 分。

7、参考答案（6 分）：



扫描全能王 创建

8、参考答案：

希尔排序（2 分）： $O(n\log_2 n)$~$O(n^2)$；

起泡排序（2 分）： $O(n^2)$；

归并排序（2 分）： $O(n\log_2 n)$。

三、 综合能力题

1、

（1）所画示意图可以用来描述图 3 中有图的存储结构（3 分）,并且，该示意图结出的数据结构可用于所提的算法（2 分）。

（2）所给算法基本思想的描述清晰（2 分），能实现所需的功能（3 分）。

2、

（1） 参考答案（共 10 分）：

第 06 行（2 分）：G->nodeList[v].label

第 09 行（2 分）：front != rear

第 11 行（2 分）：int j = Q[++front]

第 13 行（2 分）：p != NULL

第 20 行（2 分）：Q[++rear] = j;

（2）参考答案（共 8 分）：

```
#define MAXSIZE 100;
typedef char[2] DataType;
typedef struct EdgeNode        （2 分）
{
    int adjIndex;
    struct EdgeNode *next;
}
typedef struct Node            （2 分）
{
    DataType label;
    struct EdgeNode *first;
}
typedef struct ALGraph         （3 分）
{
    struct Node nodeList[MAXSIZE];
    int nodeNum, edgeNum;
}
```

（3） 参考答案（2 分）：$v_0v_1v_2v_3$，或者，$v_0v_2v_1v_3$

扫描全能王 创建

一、1. 顺序表：优点：① 顺序存储，支持随机访问
② 查找操作方便，时间、空间复杂度低
缺点：① 删除、插入操作不便，时间、空间复杂度较高
② 需要事先知道顺序表大致长度
③ 扩容一般呈2倍空间增大，易产生空间浪费

链表：优点：① 删除、插入操作方便，时间、空间复杂度低
② 不必事先知道顺序表大致长度
③ 存储空间动态变化，不会造成空间浪费

缺点：① 查找操作不便，时间、空间复杂度高
② 不支持随机访问

基于以上特点，若后续对字符串的查找操作较多，则适合用顺序表存储；
若后续对存储的字符串删除、插入操作更频繁，则适合用链表存储

2.
```
typedef int DataType;
typedef struct Node
{
    DataType data;
    struct Node* next;
}*top;
```

顶点表结点 (Vnode)

vertex	firstin	firstout

边表结点 (Edge)

toNode	fromNode	sameFrom	sameTo

NodeList

3. // 有向图十字链表
```
#define MaxSize 120
typedef char[2] DataType;
typedef struct Edge
{
    int fromNode, toNode;
    struct Edge* sameFrom, *sameTo;
}Edge;
typedef struct Node
{
    DataType vertex;
    struct Edge *firstin, *firstOut;
}Node;
typedef struct Graph
{
    struct nodeList[MaxSize];
    int vexNum, edgeNum;
}Graph;
```

// 无向图 邻接多重表

顶点表结点

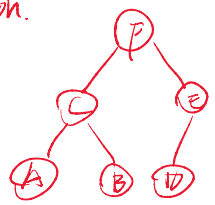
vertex	first

边表结点

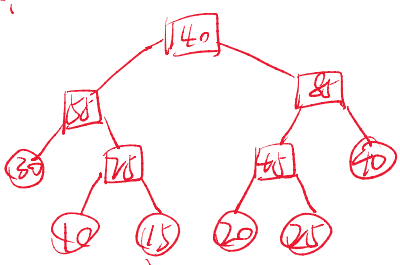
ivex	inext	jvex	jnext

```
#define MaxSize 100
typedef char DataType;
typedef struct Edge
{
    int ivex, jvex;
    struct Edge* inext, *jnext;
}Edge;
typedef struct Node
{
    DataType vertex;
    struct Edge* first;
}Node;
typedef struct Graph
{
    Struct NodeList[MaxSize];
    int edgeNum, edgeNum;
}Graph;
```

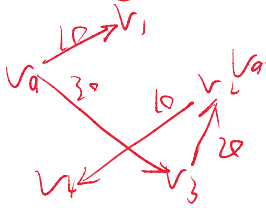
二、1.



2.



3.



$V_0 \to V_3 \to V_2 \to V_4$

长度为 30+20+10=60

4.

	V_1	V_2	V_3	V_4	V_5	V_6	V_7		
vl	0	28	36	18	4	21	7		
vl	0	28	36	18	11	27	13		
	a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
ee	0	0	11	7	7	18	18	21	28
dl	0	6	11	12	14	18	24	27	28
dl-ee	0	6	0	5	5	0	6	6	0

$V_1 \to V_5 \to V_4 \to V_2 \to V_3$

5、 ② → ① ⑤ → ② ⑤ ⑧ → ⑤ ② ⑧ → ⑤ ② ⑧ ⑦ → ⑤ ② ⑦ ⑧

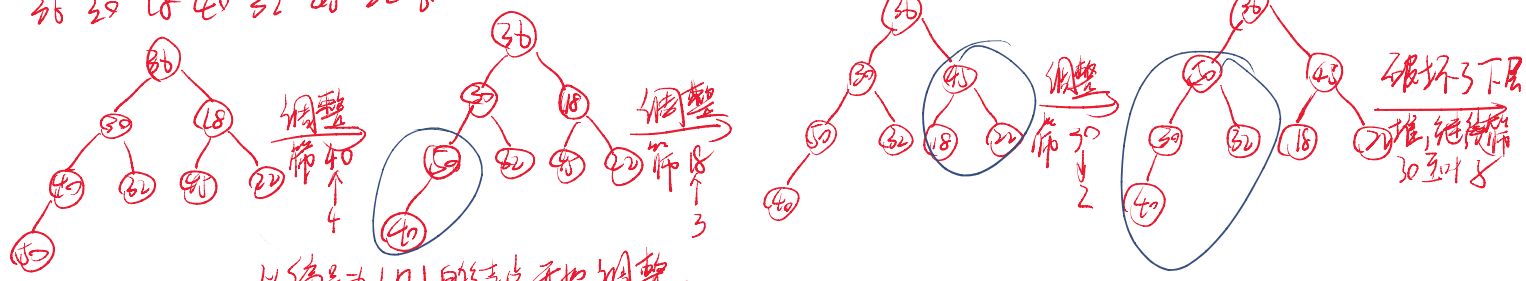
不平衡　　调整

6、① 空间利用率高：每次遍历完一条支路都需回溯，调整树内元素，此时栈内元素会相应删减，提高了空间利用率

② 使用回溯，栈顶元素总是最后访问的结点，回溯操作与栈先进后出的特点吻合

③ 递归自然：栈是一种常见的递归结构，而在深度优先遍历算法中，对每条支路的遍历和回溯都具有相似性，适合递归实现

7、 36 39 18 40 32 45 22 30

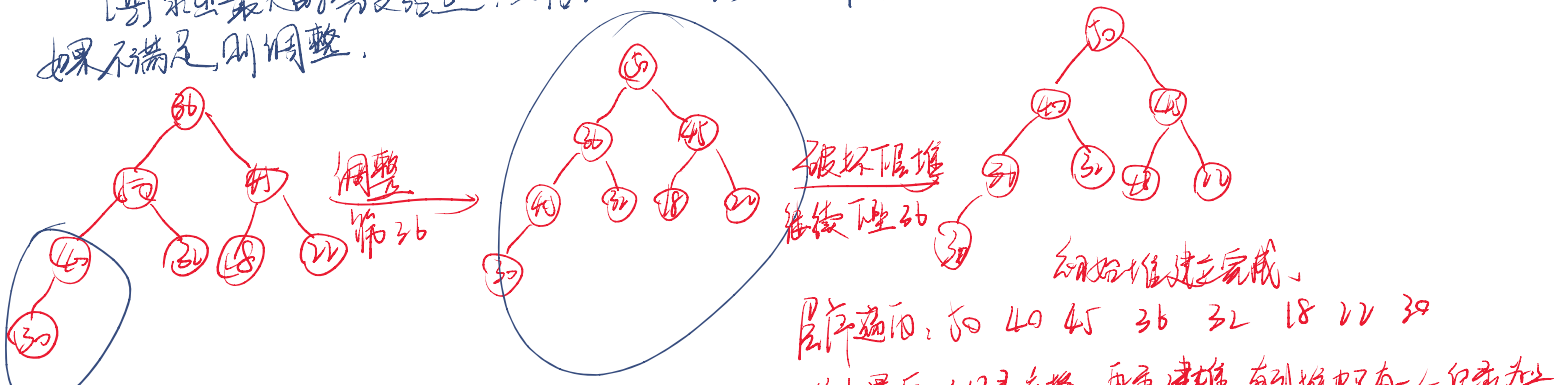


以编号为 $\lfloor \frac{n}{2} \rfloor$ 的结点开始调整

$\lfloor \frac{n}{2} \rfloor = 4$，所以 40 开始

将待调整序列建立成大根堆，

从 $\lfloor \frac{n}{2} \rfloor$ 找出最大的分支结点，然后依次以 $\lfloor \frac{n}{2} \rfloor$ 检查到序号较结点（即根），看是否满足大根堆要求，如果不满足，则调整。



初始堆建立完成。

层序遍历：50 40 45 36 32 18 22 30

堆排序：首先将记录序列调整为大根堆，然后将堆顶与堆中最后一个记录交换，再重建堆，直到堆中只有一个记录为止

8、

排序名称	时间性能（平均）	最好	最坏	空间性能
直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$
希尔	$O(n\log_2 n) \sim O(n^2)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$
起泡	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$
快速	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n) \sim O(n)$
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
堆	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$
归并	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$

二、1、




```
#define MaxSize 100
typedef char DataType;
typedef struct Edge
{ int toNode, fromNode;
  struct Edge *sameFrom, *sameTo;
}Edge;
typedef struct Node
{ DataType vertex;
  struct Edge *firstOut, *firIn;
}Node
typedef struct Graph
{ struct nodeList[MaxSize];
  int nodeNum, edgeNum;
}Graph;
```

① 从图中某顶点 v 出发开始深度优先遍历

② 从 v 的相邻访问过结点中选取一个顶点 w，然后从 w 开始深度优先遍历，若在非回溯过程中重复访问了已经访问过的顶点，则终止该回路，换路继续；到刚重复上面的过程直到图中所有与 v 相通的顶点均被访问过。